Pipette Commands

Last updated Jan 15, 2025 at 5:15 PM GMT+2

Pipette Command API

Preparation

1. Connect Pipette with PC (Micro-USB to USB-A)
2. Pipette will automatically register as Serial Interface (Windows: COMX, Linux: /dev/ttyACMX) with the name "Picus 2"
3. Start the Pipette
4. Open Serial Port to pipette (Speed: 230400, Data bits 8, Stop bits 1, No parity) or connect via Blueooth

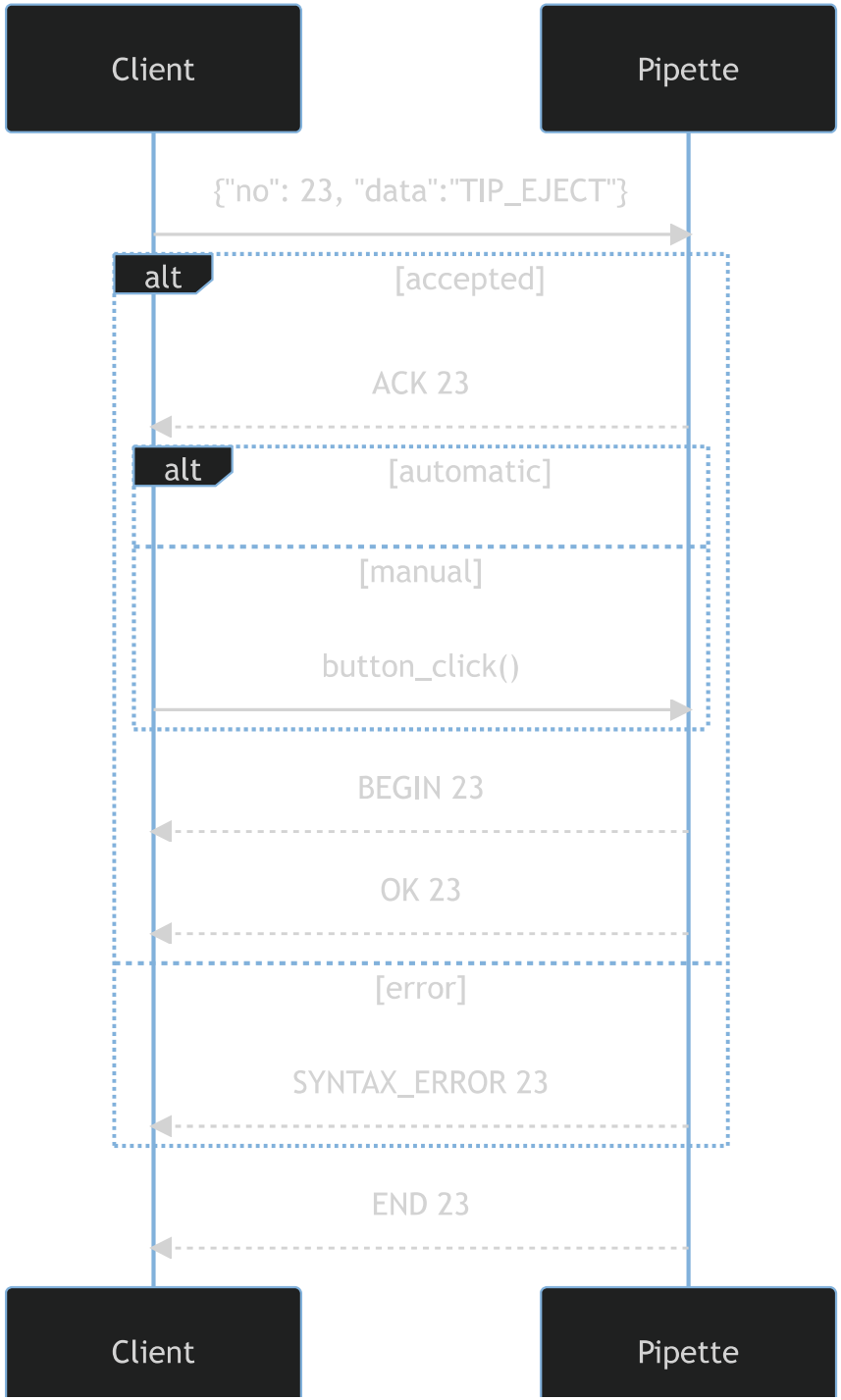
Pipette Commands

General

- Commands and corresponding responses always end with `\r\n`
- Commands are buffered
- Most commands can only be recognized when in "Protocol" mode
- Buttons are recognized everywhere and are not buffered
- Only motor commands are affected by the AUTO flag

Handshake

ACK <no>
BEGIN <no> (VERBOSE only)
<Response> (only if return value)
<RESULT> OK or Error
END <no> (VERBOSE only)


Client
Pipette
{"no": 23, "data":"TIP_EJECT"}
alt
[accepted]
ACK 23
alt
[automatic]
[manual]
button_click()
BEGIN 23
OK 23
[error]
SYNTAX_ERROR 23
END 23
Client
Pipette

List of button commands

- Picus can be controlled remotely by emitting button events via the command interface.
- Button commands do not return results

Command	Example	Description
TRIGGER_BUTTON_POWER	`{"button": "TRIGGER_BUTTON_POWER"}`	Emulates Power Button press.
TRIGGER_BUTTON_LEFT	`{"button": "TRIGGER_BUTTON_LEFT"}`	Emulates Left (Back) Button press.
TRIGGER_BUTTON_MIDDLE	`{"button": "TRIGGER_BUTTON_MIDDLE"}`	Emulates Middle (Edit) Button press.
TRIGGER_BUTTON_RIGHT	`{"button": "TRIGGER_BUTTON_RIGHT"}`	Emulates Right (OK) Button press.
TRIGGER_BUTTON_TOP	`{"button": "TRIGGER_BUTTON_TOP"}`	Emulates Top (Accept/Run) Button press.
TRIGGER_BUTTON_PRESET	`{"button": "TRIGGER_BUTTON_PRESET"}`	Emulates Preset (Memory) Button press.
TRIGGER_BUTTON_TIPEJECT	`{"button": "TRIGGER_BUTTON_TIPEJECT"}`	Emulates Tip-Eject Button press.
UP	`{"button": "UP"}`	Emulates Scrollwheel Up movement.
DOWN	`{"button": "DOWN"}`	Emulates Scrollwheel Down movement.

Multi-button command BUTTON_SEQUENCE

Requires payload in list format with "command-delay" pairs: "payload": ["TRIGGER/RELEASE_BUTTON_XX", YY, ...]

Command	Examples	Description	Factory Mode only
Random sequence	`{"data": "BUTTON_SEQUENCE", "payload": ["TRIGGER_BUTTON_POWER", 100, "RELEASE_BUTTON_POWER", 200, "TRIGGER_BUTTON_TOP", 150, "RELEASE_BUTTON_TOP", 250]}`	Emulates power button press for 100ms with release and delay for 200ms and then doing top button press by holding for 150ms and then release with delay after for 250ms.	yes
Double click	`{"data": "BUTTON_SEQUENCE", "payload":["TRIGGER_BUTTON_TOP", 100, "RELEASE_BUTTON_TOP", 100, "TRIGGER_BUTTON_TOP", 100, "RELEASE_BUTTON_TOP", 100]}`	Emulates top button double press with 100ms delays in between the steps.	yes
Multi-press (press at the same time)	`{"data": "BUTTON_SEQUENCE", "payload":["TRIGGER_BUTTON_TOP", 100, "TRIGGER_BUTTON_LEFT", 100, "RELEASE_BUTTON_TOP", 100, "RELEASE_BUTTON_LEFT", 100]}`	Emulates pressing top- and left-buttons at the same time with 100ms in between the steps.	yes

List of Commands

Command	Parameters	Example	Description	Returns	Buffered
JSON_FORMAT<value>	value: 1 = Response in Json 0 = Simple response (Default)	`{"no": 0, "data":"JSON_FORMAT 1"}`	Defines, if the response comes back in Json format	Result	no
AUTO<value>	value: 1 = automatic execution 0 = manual execution	`{"no": 0, "data":"AUTO 1", "gui":"AUTO"}`	NoAutomatic command execution	Result	no
VERBOSE <value>	value: 0 = silent mode 1 = verbose mode	`{"no": 0, "data":"VERBOSE 1", "gui":"VERBOSE"}`	Switch between verbose and silent handshake mode	Result	no
GET_VERSION	N/A	`{"no": 0, "data":"GET_VERSION"}`	Returns Firmware version (e.g. CP-0.1)	Response	no
SET_BACKLIGHT<0-20>	0 <= <value> >=20	`{"no": 0, "data":"SET_BACKLIGHT 15"}`		Result	no
REBOOT	N/A	`{"no": 0, "data":"REBOOT"}`	Reboot pipette (Standby)	Result	yes
RESET	N/A	`{"no": 0, "data":"RESET"}`	Formats the data flash and reboots pipette. This is a PRODUCTION reset, which erases all data,	Result	yes

Command	Parameters	Example	Description	Returns	Buffered
			including the serial number. Additionally, after this command, the pipette enters TEST mode (pipette does not reboot after reset and is unresponsive to button presses), requiring a simultaneous power button+operation button long press (about 5 seconds) in order to power on the pipette		
ERASE_DATA <data>	data: USER or FACTORY or PRODUCTION	`{"no": 0, "data":"ERASE_DATA USER"}`	Erases given data and reboots pipette	Result	yes
SET_BT_MAC <mac>		`{"no": 0, "data":"SET_BT_MAC 123456789AFF"}`	Set MAC address for Bluetooth	Response	no
SET_SERIAL <serial>	Serial No :<int>	`{"no": 0, "data":"SET_SERIAL 1237456"}`	Set serial number	Result	no

Command	Parameters	Example	Description	Returns	Buffered
GET_SERIAL		`{"no": 0, "data":"GET_SERIAL"}`	Returns serial number	Response	no
GET_ID		`{"no": 0, "data":"GET_ID"}`	Returns pipette id	Response	no
GET_BATTERY_LEVEL		`{"no": 0, "data":"GET_BATTERY_LEVEL"}`	Returns battery battery level	Response	no
GET_BATTERY_DATA		`{"no": 0, "data":"GET_BATTERY_DATA"}`	Returns battery data: voltage, current and charger's status	Response	no
GET_MIN_VOLUME		`{"no": 0, "data":"GET_MIN_VOLUME"}`	Returns minimum volume	Response	no
GET_NOMINAL_VOLUME		`{"no": 0, "data":"GET_NOMINAL_VOLUME"}`	Returns nominal volume	Response	no
GET_MODEL		`{"no": 0, "data":"GET_MODEL"}`	Returns pipette model	Response	no
GET_TIME		`{"no": 0, "data":"GET_TIME"}`	Returns time	Response	no
GET_DATE		`{"no": 0, "data":"GET_DATE"}`	Returns date	Response	no
GET_STATUS_CALIBRATION		`{"no": 0, "data":"GET_STATUS_CALIBRATION"}`	Returns calibration status	Response	no

Command	Parameters	Example	Description	Returns	Buffered
GET_STATUS_MAINTENANCE		`{"no": 0, "data":"GET_STATUS_MAINTENANCE"}`	Returns maintenance status	Response	no
GET_STATUS_CHECK		`{"no": 0, "data":"GET_STATUS_CHECK"}`	Returns quick check status	Response	no
SET_CAL_DATA <model> <p0> <p1> <p2> <p3> <d0> <d1> <d2> <d3>	model: SINGLE_CHANNEL_10UL, SINGLE_CHANNEL_10UL_V2, SINGLE_CHANNEL_120UL, SINGLE_CHANNEL_300UL, SINGLE_CHANNEL_1000UL, SINGLE_CHANNEL_5ML, SINGLE_CHANNEL_10ML, MULTI_CHANNEL_10UL, MULTI_CHANNEL_10UL_V2, MULTI_CHANNEL_120UL, MULTI_CHANNEL_300UL, MULTI_CHANNEL_1200UL p/d :<int16_t>	`{"no": 0, "data":"SET_CAL_DATA SINGLE_CHANNEL_1000UL 1 2 3 4 5 6 7 8"}`	Set calibration points steps	Result	no
GET_CAL_DATA <model>	model: SINGLE_CHANNEL_10UL, SINGLE_CHANNEL_10UL_V2, SINGLE_CHANNEL_120UL, SINGLE_CHANNEL_300UL, SINGLE_CHANNEL_1000UL, SINGLE_CHANNEL_5ML, SINGLE_CHANNEL_10ML,	`{"no": 0, "data":"GET_CAL_DATA SINGLE_CHANNEL_1000UL"}`	Get calibration points steps	Response	no

Command	Parameters	Example	Description	Returns	Buffered
	MULTI_CHANNEL_10UL, MULTI_CHANNEL_10UL_V2, MULTI_CHANNEL_120UL, MULTI_CHANNEL_300UL, MULTI_CHANNEL_1200UL				
GET_LATEST_LOG		`{"no": 0, "data":"GET_LATEST_LOG"}`	Returns the log file containing latest entries. Result is FAILED if no logs are available.	Response	no
GET_NEXT_LOG		`{"no": 0, "data":"GET_NEXT_LOG"}`	Returns the log file containing next latest entries. Result is FAILED if no more older logs are available.	Response	no
ENABLE_IMPORT_EXPORT <0/1>		`{"no": 0, "data":"ENABLE_IMPORT_EXPORT 1"}`	Enables import/export of pipette data	Result	no
IMPORT <KEY 1*> <PAYLOAD>		`{"no": 0, "data":"IMPORT Settings/Theme", "payload":"VIOLET"}`	Imports Theme settings	Result	no

Command	Parameters	Example	Description	Returns	Buffered
EXPORT <KEY 2*>		`{"no": 0, "data":"EXPORT Settings/Theme"}`	Gets Theme settings	Result	no
SHOW_SERIAL		`{"no": 0, "data":"SHOW_SERIAL"}`	Shows serial number	Result	no
HIDE_SERIAL		`{"no": 0, "data":"HIDE_SERIAL"}`	Hide serial number	Result	no
CLOSE_PROD		`{"no": 0, "data":"CLOSE_PROD"}`	Clear FactoryMode flag	Result	no
TEST		`{"no": 0, "data":"TEST"}`	Set TestMode flag. Test mode can be overrun by pressing **power** and **top** buttons at the same time when rebooting	Result	no
CAL_FUEL_GAUGE		`{"no": 0, "data":"CAL_FUEL_GAUGE"}`	Start fuel gauge calibration. Pipette is reboted at the end of the calibration.	Result	no
ENABLE_MOTOR_CONTROL <0/1>		`{"no": 0, "data":"ENABLE_MOTOR_CONTROL 1"}`		Result	no

Command	Parameters	Example	Description	Returns	Buffered
GET_ENCODER		`{"no": 0, "data":"GET_ENCODER"}`	Returns the absolute encoder value	Response	OK or NOT_ALLOWED
SCREENSHOT	GIF = Returns a Gif with a color palete of 4 colors, GIF_4BIT = Returns a Gif with a color palete of 16 colors, BMP = Returns a Bitmap, BMP_RLE8= Returns RLE8 compressed Bitmap, <empty> = Same as GIF	`{"no": 0, "data":"SCREENSHOT GIF"}`	Returns a Screenshot as Base64 Encoded String	Response	
TOUCH		`{"no": 0, "data":"TOUCH"}`	Simulate device operation	Response	
CLEAR_MOTOR_CMD_QUEUE		`{"no": 0, "data":"CLEAR_MOTOR_CMD_QUEUE"}`	Abort current pending motor command and clear motor command queue	Response	
GET_DATE_TIME		`{"no": 0, "data":"GET_DATE_TIME"}`	Returns the current date and time in ISO format (YYYY-MM-DD hh:mm:ss)	Response	

Command	Parameters	Example	Description	Returns	Buffered
SET_DATE_TIME <YYYY-MM-DD> <hh:mm:ss>		`{"no": 0, "data":"SET_DATE_TIME 2023-05-09 16:00:00"}`	Sets the current date and time.	Response	
SET_IMAGE <screen> <id>	ID is image id (1,2 etc) or 0 for no image	`{"no": 0, "data":"SET_IMAGE STARTUP 1"}`		Response	

1*) **KEY** can be one of the following:

- "Reminders/Calibration", "Reminders/Maintenance", "Reminders/QuickCheck"
- "Modes/Diluting", "Modes/Manual", "Modes/MultiAspirating", "Modes/MultiDispensing", "Modes/Pipetting", "Modes/ReversePipetting", "Modes/SeqDispensing", "Modes/Titrating"
- "Settings/Backlight", "Settings/Bluetooth", "Settings/Language", "Settings/Sound", "Settings/Theme", "Settings/TipEjection", "Settings/UserId", "Settings/UserAdjustment", "Settings/UserAdjustmentSlots"
- **NOTE** certain data cannot be imported to the pipette; these include Factory and Production data

2*) **KEY** can be one of the following:

- "Reminders/Calibration", "Reminders/Maintenance", "Reminders/QuickCheck"
- "Modes/Diluting", "Modes/Manual", "Modes/MultiAspirating", "Modes/MultiDispensing", "Modes/Pipetting", "Modes/ReversePipetting", "Modes/SeqDispensing", "Modes/Titrating"
- "Settings/Backlight", "Settings/Bluetooth", "Settings/Language", "Settings/Sound", "Settings/Theme", "Settings/TipEjection", "Settings/UserId", "Settings/UserAdjustment", "Settings/UserAdjustmentSlots"
- "Factory/Adjustment", "Factory/Pipette", "Factory/DurabilityTest"
- "Production/SerialNumber", "Production/MotorLedLevel", "Production/FuelGaugeConfigID", "Production/BluetoothMac", "Production/CalibrationPoints"

List of Motor Commands

In order to be able to execute motor commands, this must first be enabled with the ENABLE_MOTOR_CONTROL command.

Gui tags

Each motor command can have the following "gui_*"-tag, that allows writing a message into the Pipette's screen.

gui_pre: Shown after command is acknowledged / default: is command instruction
gui_while: Shown while command execution / default depends on command
gui_after: Shown after command execution (this will remain) / default : ""

- GUI Commands like WRITE_DISPLAY and WRITE_QR will be overwritten by and gui command
- Example: {"no": 0, "data":"NO_ACTION", "gui_pre":"Hello\nWorld!"}

Some motor commands can have the following "position" tag, that allows to set a position indicator:

```
"position":
{
   "title":"Count",
   "value":"123"
}
```

The "title" tag allows to set a custom title text up to 5 chars long.
The "value" tag allows to set a custom value text up to 4 chars long.

RUN_MIX, RUN_ASPIRATE, and RUN_DISPENSE have also "gui_volume" tag, which takes a number as parameter. This will override the volume on the screen, but won't affect the actual volume that is aspirated/dispensed. Valid range is from 0 to maximum (nominal) volume of the pipette, and the unit is the model's default unit. Values outside this will return ERR_RANGE_PARAMETERS. Setting the volume to 0, will hide the volume on the screen completely.

Command	Parameters	Example	Description	Returns	Buffered	Tracker	Factory Mode only
NO_ACTION	N/A	`{"no": 0, "data":"NO_ACTION", "gui_pre":"NO_ACTION"}`	Ping to check parser readiness	Result	no	no	no
HOME	N/A	`{"no": 0, "data":"HOME", "gui_pre":"HOME"}`	Run to home position	Result	yes	no	no
PWM <value>	value: -533...533	`{"no": 0, "data": "PWM 100"}`	Set motor PWM	Result	yes	no	yes
CURRENT <measurements> <delay>	measurements = count for cycles, delay = in between in ms	`{"no": 0, "data": "CURRENT 10 500"}`	Log current measurements, output: V[value]	Result	yes	no	yes
BLOW_OUT<go_home> <speed> <delay_ms>	go_home: 1/0 speed: 1...9 delay_ms: delay for go_home	`{"no": 0, "data": "BLOW_OUT 1 1 3000", "gui_pre":"BLOW_OUT"}`	Perform blowout	Result	yes	no	no
RUN_INIT	N/A	`{"no": 0, "data":"RUN_INIT", "gui_pre":"RUN_INIT"}`	Pipette initialization (find home position)	Result	yes	no	no
TIP_EJECT	N/A	`{"no": 0, "data":"TIP_EJECT", "gui_pre":"TIP_EJECT"}`	Performs tip ejection	Result	yes	no	
RUN_MIX <cycles> <speed> <amount> <excess (optional)>	0 <= <amount> <= mixable amount	`{"no": 0, "data":"RUN_MIX 5 7 100 2", "gui_pre":"RUN_MIX", "position":{"title":"Track", "value":"A 1"}}`	Mix specific amount in uL	Result	yes	yes	no

Command	Parameters	Example	Description	Returns	Buffered	Tracker	Factory Mode only
	1 <= <speed> <= 9						
RUN_ASPIRATE <amount> <speed>	min volume <= <amount> <= aspiratable amount 1 <= <speed> <= 9	`{"no": 0, "data":"RUN_ASPIRATE 500 7", "gui_pre":"RUN_ASPIRATE", "position":{"title":"Count", "value":"123"}}`	Aspirate specific amount in uL	Result	yes	yes	no
RUN_DISPENSE <amount> <speed>	min volume <= <amount> <= dispensible amount 1 <= <speed> <= 9	`{"no": 0, "data":"RUN_DISPENSE 1 7", "gui_pre":"RUN_DISPENSE", "position":{"title":"Tube", "value":"X"}, "gui_volume":100}`	Dispense specific amount in uL	Result	yes	yes	no
BRAKE <open>	open: 1/0	`{"no": 0, "data": "BRAKE 1"}`	Set brake open or closed (1/0)	Result	yes	no	yes

Command Results

General

Result	Description
OK	Command execution was successful
FULL	Command buffer is full, will be ignored
SYNTAX_ERROR	Command has syntactical errors
ERROR_PARSING	Command payload could not be parsed
UNKNOWN_COMMAND	Command is unknown
MISSING_PARAMETERS	Command is missing input parameters
ERR_RANGE_PARAMETERS	Parameter value is outside valid range
CHK_ERROR	Firmware data is corrupt
NOT_ALLOWED	Command is currently not allowed
FAILED	Unexpected error
MOTOR_CONTROL_ABORTED	Motor control aborted by user

Motor Errors

Result	Description
INVALID_PARAMETERS	Command parameters are invalid
TIP_EJECT_ERROR	Error in tip ejection process
RUN_TO_ZERO_ERROR	Error in initialization process
UNDERSTEP_OVERSTEP_ERROR	Motor target position could not be reached
FATAL_ERROR	Motor has undefined errors and needs reset